



RAPPORT DE STAGE

DIPLÔME BACHELOR EN TECHNOLOGIE

FILIÈRE : GÉNIE LOGICIEL (GL)

Conception et Développement d'une Plateforme
Intelligente de Présélection des Candidats

Réalisé par :

SAMRANI OUAHID

Soutenu devant le jury composé de :

- M. YOUNES AZIZ BACHIRI (Encadrant académique)
- M. DAHBI MAROUANE (Encadrant entreprise)
- M. ABDELHAY ZOIZOU (Membre du jury)
- M. NAJIB NFISSI (Membre du jury)

Lieu de stage : ENSA Béni Mellal — USMS

Période : Du 06/04/2026 au 13/06/2026

Année Universitaire : 2025–2026

Remerciements

Avant d'entamer la présentation de ce travail, je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué à la réussite de ce stage.

Je remercie tout d'abord mon encadrant professionnel au sein de l'ENSA Béni Mellal, **M. DAHBI MAROUANE**, pour son encadrement rigoureux, ses orientations pertinentes et sa disponibilité tout au long de la période de stage.

J'adresse également mes sincères remerciements à mon encadrant académique à l'EST de Fès, **M. YOUNES AZIZ BACHIRI**, pour son suivi pédagogique et ses recommandations précieuses.

Ma reconnaissance va aussi à l'ensemble du personnel du service de la Scolarité de l'ENSA BM, qui a facilité mon intégration et m'a fourni les informations nécessaires à la compréhension du processus de présélection.

Je remercie les membres du jury, **M. ABDELHAY ZOIZOU** et **M. NAJIB NFISSI**, d'avoir accepté d'évaluer ce travail.

Enfin, je remercie l'USMS et l'EST de Fès pour la qualité de la formation dispensée en Génie Logiciel.

Résumé

Le présent rapport rend compte du stage de fin d'études effectué au sein de l'ENSA Béni Mellal, rattachée à l'Université Sultan Moulay Slimane. Ce stage s'inscrit dans le cadre de l'obtention du Bachelor Universitaire de Technologie en Génie Logiciel de l'EST de Fès.

Le projet consiste en la conception et le développement d'une plateforme web intelligente dédiée à la présélection des candidats aux filières d'ingénieur de l'ENSA BM. Cette plateforme automatise le traitement des dossiers de candidature, depuis le dépôt en ligne jusqu'au classement final, en passant par la vérification OCR des documents, le scoring académique pondéré par filière et la gestion des épreuves écrites.

La solution repose sur une architecture Full Stack moderne : Django REST Framework pour le backend et React 18 avec Tailwind CSS pour le frontend. Elle intègre un moteur de règles de rejet automatique (7 types), un algorithme de scoring avec fuzzy matching des matières, un système de notifications en temps réel via WebSocket, et un module d'import des notes d'épreuve écrite au format Excel.

Mots-clés : Présélection, ENSA, Django, React, Scoring automatique, OCR, Fuzzy Matching, WebSocket, LMD, Full Stack.

Abstract

This report presents the internship carried out at the National School of Applied Sciences of Béni Mellal (ENSA BM), part of Sultan Moulay Slimane University. The project involves the design and development of an intelligent web platform for candidate preselection to ENSA BM engineering programs.

The solution is built on a modern Full Stack architecture combining Django REST Framework for the backend and React 18 with Tailwind CSS for the frontend. It features an automatic rejection rule engine (7 types), a weighted scoring algorithm with fuzzy matching, real-time WebSocket notifications, and an Excel-based exam score import module.

Keywords : Preselection, ENSA, Django, React, Automatic scoring, OCR, Fuzzy Matching, WebSocket, LMD, Full Stack.

Table des matières

Remerciements	i
Résumé	ii
Abstract	iii
Liste des abréviations	ix
Introduction générale	1
1 Contexte du Projet	2
1.1 Introduction	2
1.2 Présentation de l'organisme d'accueil	2
1.2.1 L'ENSA Béni Mellal	2
1.2.2 Missions et offre de formation	2
1.2.3 Service de la Scolarité	2
1.3 Contexte et problématique	3
1.3.1 Le processus actuel de présélection	3
1.3.2 Problèmes identifiés	3
1.3.3 Le système LMD marocain	3
1.4 Objectifs du projet	4
1.4.1 Objectif principal	4
1.4.2 Objectifs spécifiques	4
1.5 Périmètre fonctionnel	4
1.6 Méthodologie de travail	4
1.7 Conclusion	5
2 Architecture et Base de données	6
2.1 Introduction	6
2.2 Architecture globale	6
2.2.1 Vue d'ensemble	6
2.2.2 Architecture REST stateless	7
2.3 Stack technologique	8
2.4 Organisation modulaire du backend	8
2.5 Dictionnaire des modèles Django	8
2.5.1 Application Users	10
2.5.2 Application Candidatures	10
2.5.3 Application Administration et Scoring	10
2.6 Relations et cardinalités	13
2.7 Machine d'états du dossier	13

2.8	Conclusion	13
3	Logique Métier et Algorithmes	15
3.1	Introduction	15
3.2	Moteur de règles de rejet automatique	15
3.2.1	Principe de fonctionnement	15
3.2.2	Types de règles implémentés	15
3.2.3	Journalisation des rejets	16
3.3	Algorithme de scoring pondéré par filière	16
3.3.1	Principe général	16
3.3.2	Coefficients des quatre filières	16
3.3.3	Formule de calcul du score dossier	16
3.3.4	Bonus de mention et coefficient diplôme	17
3.3.5	Formule du score final	18
3.4	Pipeline OCR et Fuzzy Matching	18
3.4.1	Extraction OCR	18
3.4.2	Nettoyage du bruit OCR	19
3.4.3	Fuzzy Matching des matières	19
3.4.4	Scoring avec fallback	19
3.5	Calcul automatique des mentions	19
3.6	Conclusion	20
4	Implémentation, Asynchronisme et Tests	21
4.1	Introduction	21
4.2	Interfaces développées	21
4.3	Gestion de l'asynchronisme	21
4.3.1	Problématique	21
4.3.2	Solution : threading.Thread	24
4.3.3	Gestion des connexions base de données	24
4.4	Système de notifications en temps réel	24
4.4.1	Architecture WebSocket	24
4.4.2	Envoi en masse avec progression	25
4.5	Système de logging	25
4.6	Sécurité de l'application	25
4.7	Stratégie de tests	27
4.7.1	Niveaux de tests	27
4.7.2	Tests du moteur de règles	27
4.7.3	Tests de l'algorithme de scoring	27
4.7.4	Récapitulatif des tests	27
4.7.5	Couverture de code	27
4.8	Difficultés rencontrées et solutions	27
4.9	Conclusion	27

Conclusion générale	28
Perspectives d'amélioration	28
Bibliographie	28

Table des figures

2.1	Diagramme de cas d'utilisation global de la plateforme	6
2.2	Architecture générale de la plateforme de présélection	7
2.3	Organisation modulaire du backend Django	9
2.4	Diagramme de classes de la plateforme	11
2.5	Machine d'états du dossier de candidature	14
3.1	Fonctionnement du moteur de rejet automatique	16
3.2	Processus de calcul du score pondéré	17
3.3	Pipeline OCR et traitement des relevés de notes	18
4.1	Interface d'accueil de la plateforme	22
4.2	Formulaire candidature	22
4.3	Interface de dépôt des documents justificatifs	22
4.4	Tableau de bord du responsable de présélection	23
4.5	Consultation détaillée d'un dossier candidat	23
4.6	Configuration des filières et règles de scoring	23
4.7	Tableau de bord analytique du responsable de présélection	24

Liste des tableaux

1.1	Fiche signalétique de l'ENSA BM	2
1.2	Filières d'ingénieur de l'ENSA BM	3
1.3	Seuils des mentions LMD	3
1.4	Périmètre fonctionnel du projet	4
1.5	Planning prévisionnel du projet	5
2.1	Stack technologique du projet	8
2.2	Applications Django du backend	8
2.3	Services métier du module candidatures	9
2.4	Modèle User	10
2.5	Modèle Candidat	10
2.6	Modèle Dossier	11
2.7	Modèle Document	11
2.8	Modèle NoteSemestre	11
2.9	Modèle Filiere	12
2.10	Modèles d'administration et scoring	12
2.11	Relations et cardinalités du modèle de données	12
2.12	Machine d'états du dossier de candidature	13
3.1	Les 7 types de règles de rejet automatique	15
3.2	Matrice des pondérations par filière	16
3.3	Bonus de mention	17
3.4	Coefficients par type de diplôme	18
3.5	Corrections automatiques du bruit OCR	19
4.1	Interfaces développées par rôle	21
4.2	Configuration des loggers	25
4.3	Mesures de sécurité implémentées	25
4.4	Niveaux de tests	26
4.5	Tests du moteur de règles	26
4.6	Tests de l'algorithme de scoring	26
4.7	Récapitulatif des résultats de tests	26
4.8	Couverture de code par module	26
4.9	Difficultés rencontrées et solutions	27

Liste des abréviations

Abréviation	Signification
API	Application Programming Interface
CIN	Carte d'Identité Nationale
CNE	Code National de l'Étudiant
CSRF	Cross-Site Request Forgery
DRF	Django REST Framework
ENSA	École Nationale des Sciences Appliquées
EST	École Supérieure de Technologie
G2ER	Génie de l'Énergie et Énergies Renouvelables
IAA	Ingénierie Agro-Alimentaire
IACS	Informatique Appliquée et Cybersécurité
JWT	JSON Web Token
KPI	Key Performance Indicator
LMD	Licence — Master — Doctorat
OCR	Optical Character Recognition
ORM	Object-Relational Mapping
RBAC	Role-Based Access Control
REST	Representational State Transfer
SMTP	Simple Mail Transfer Protocol
SPA	Single Page Application
TDI	Télécommunications et Développement Informatique
USMBA	Université Sidi Mohamed Ben Abdellah
USMS	Université Sultan Moulay Slimane
XSS	Cross-Site Scripting

Introduction générale

Dans un contexte de transformation numérique des établissements d'enseignement supérieur au Maroc, la gestion des processus d'admission et de présélection constitue un enjeu majeur pour les écoles d'ingénieurs. L'ENSA Béni Mellal accueille chaque année un nombre croissant de candidatures pour ses filières d'ingénieur, tant au niveau Bac+2 qu'au niveau Bac+3.

Le processus actuel de présélection, principalement manuel et basé sur des fichiers Excel, présente plusieurs limites : risque d'erreurs humaines, lenteur du traitement, absence de traçabilité et difficulté de gestion des volumes importants de dossiers. Le service de la Scolarité a exprimé le besoin d'une solution informatique capable d'automatiser et de fiabiliser ce processus.

C'est dans ce contexte que s'inscrit notre projet de stage : la conception et le développement d'une plateforme web intelligente de présélection des candidats. Ce rapport est structuré en quatre chapitres :

- **Chapitre 1 — Contexte du Projet** : présentation de l'organisme d'accueil, problématique et objectifs.
- **Chapitre 2 — Architecture et Base de données** : architecture Full-Stack, stack technologique et dictionnaire des modèles Django.
- **Chapitre 3 — Logique Métier et Algorithmes** : moteur de règles, scoring pondéré, pipeline OCR et fuzzy matching.
- **Chapitre 4 — Implémentation, Asynchronisme et Tests** : interfaces, threading, notifications WebSocket, sécurité et validation.

Chapitre 1

Contexte du Projet

1.1 Introduction

Ce chapitre présente le cadre général dans lequel s’inscrit notre stage de fin d’études. Nous y décrivons l’organisme d’accueil, le contexte institutionnel, la problématique identifiée et les objectifs du projet.

1.2 Présentation de l’organisme d’accueil

1.2.1 L’ENSA Béni Mellal

L’École Nationale des Sciences Appliquées de Béni Mellal (ENSA BM) est un établissement public d’enseignement supérieur rattaché à l’Université Sultan Moulay Slimane (USMS). Créée en 2003 par décret ministériel n° 2-03-200, elle fait partie du réseau des douze ENSA marocaines et forme des ingénieurs d’État dans les domaines des sciences appliquées.

Tableau 1.1 – Fiche signalétique de l’ENSA BM

Rubrique	Information
Dénomination	École Nationale des Sciences Appliquées de Béni Mellal
Sigle	ENSA BM
Université	Université Sultan Moulay Slimane (USMS)
Statut	Établissement public d’enseignement supérieur
Création	2003 — Décret n° 2-03-200
Effectif étudiants	~2 500 (2024-2025)
Enseignants	~120 enseignants-chercheurs

1.2.2 Missions et offre de formation

L’ENSA BM a pour mission de former des ingénieurs d’État polyvalents, de développer la recherche scientifique et l’innovation technologique, et de contribuer au développement socio-économique de la région Béni Mellal-Khénifra.

L’école propose quatre filières d’ingénieur accessibles aux niveaux Bac+2 et Bac+3 :

1.2.3 Service de la Scolarité

Le service de la Scolarité gère l’ensemble des processus administratifs liés aux étudiants : inscriptions, examens, stages, diplomation et présélection des candidats au cycle

Tableau 1.2 – Filières d'ingénieur de l'ENSA BM

Code	Filière	Places
TDI	Télécommunications et Développement Informatique	30
IACS	Informatique Appliquée et Cybersécurité	30
IAA	Ingénierie Agro-Alimentaire	30
G2ER	Génie de l'Énergie et Énergies Renouvelables	30

ingénieur. C'est le principal bénéficiaire et commanditaire de notre plateforme.

1.3 Contexte et problématique

1.3.1 Le processus actuel de présélection

Le processus actuel de présélection à l'ENSA BM se déroule en plusieurs étapes manuelles : (1) réception des dossiers physiques ou par email, (2) vérification manuelle de la conformité des pièces, (3) saisie des notes dans un fichier Excel, (4) tri et classement manuel, (5) convocation à l'épreuve écrite, et (6) saisie des notes d'épreuve et classement final. Ce processus mobilise plusieurs agents pendant plusieurs semaines.

1.3.2 Problèmes identifiés

- Traitement lent et chronophage : plusieurs semaines pour quelques centaines de dossiers
- Risque élevé d'erreurs de saisie lors de la transcription manuelle des notes
- Absence totale de traçabilité des décisions prises
- Manque de transparence : les candidats n'ont aucune visibilité sur l'état de leur dossier
- Doublons non détectés : difficulté d'identifier les candidatures multiples
- Classement approximatif : calculs manuels sujets à erreur

1.3.3 Le système LMD marocain

Le système LMD marocain structure les études supérieures en trois cycles. Chaque semestre est évalué sur 20 points. Les mentions sont attribuées selon les seuils suivants :

Tableau 1.3 – Seuils des mentions LMD

Moyenne	Mention
≥ 16.00	Très Bien
≥ 14.00	Bien
≥ 12.00	Assez Bien
≥ 10.00	Passable

Notre plateforme doit impérativement respecter ces seuils pour le calcul automatique des mentions et du scoring académique.

1.4 Objectifs du projet

1.4.1 Objectif principal

Concevoir et développer une plateforme web complète permettant d'automatiser et de fiabiliser le processus de présélection des candidats aux filières d'ingénieur de l'ENSA BM.

1.4.2 Objectifs spécifiques

- Automatisation du rejet des dossiers non conformes via un moteur de règles configurable
- Scoring académique pondéré par filière avec fuzzy matching des matières
- Classement automatique des candidats intégrant dossier (40%) et épreuve écrite (60%)
- Notifications en temps réel via WebSocket et email SMTP
- Traçabilité et audit complet du processus de présélection
- Import des notes d'épreuve écrite au format Excel

1.5 Périmètre fonctionnel

Tableau 1.4 – Périmètre fonctionnel du projet

Fonctionnalité	Inclus
Inscription et authentification JWT	Oui
Dépôt de dossier multi-étapes	Oui
Moteur de règles de rejet automatique	Oui
Scoring pondéré par filière	Oui
Dashboard responsable avec KPIs	Oui
Import notes Excel	Oui
Notifications WebSocket + SMTP	Oui
Export résultats Excel	Oui
Application mobile native	Non (hors scope)
Intégration Massar	Non (hors scope)

1.6 Méthodologie de travail

Nous avons adopté une approche agile itérative, organisée en sprints hebdomadaires sur 4 semaines. Chaque sprint produit un incrément fonctionnel testé et validé.

Tableau 1.5 – Planning prévisionnel du projet

Semaine	Phase	Livrables
S1	Fondations Backend	Modèles Django, API REST, Auth JWT
S2	Intelligence métier	Moteur de règles, Scoring, OCR
S3	Frontend React	Pages candidat, Dashboard, Formulaires
S4	Tests et finalisation	Tests, Corrections, Documentation

1.7 Conclusion

Ce chapitre a permis de situer le cadre institutionnel de notre stage et de définir clairement la problématique et les objectifs du projet. Le chapitre suivant présentera l'architecture technique et le modèle de données de la plateforme.

Chapitre 2

Architecture et Base de données

2.1 Introduction

Ce chapitre détaille l'architecture technique de la plateforme, le choix des technologies, l'organisation modulaire du code et le dictionnaire complet des modèles Django.

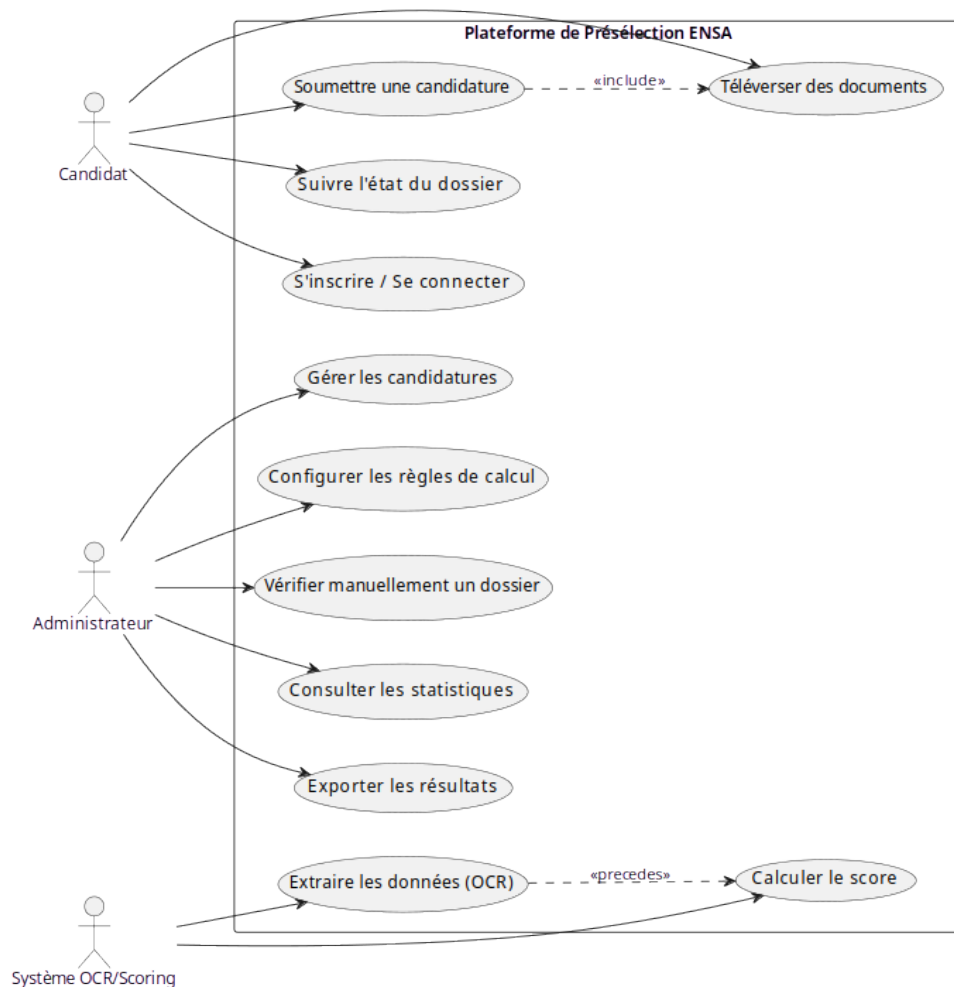


Fig 2.1 – Diagramme de cas d'utilisation global de la plateforme

2.2 Architecture globale

2.2.1 Vue d'ensemble

L'application repose sur une architecture moderne Full-Stack Client-Serveur à trois niveaux :

- Frontend : React 18 compilé via Vite, avec Tailwind CSS pour le rendu responsive. L'architecture s'appuie sur des composants réutilisables, React Router v6 et un state management optimisé via Context API.
- Backend : Django 4.2 LTS et Django REST Framework (DRF), exposant une API REST sécurisée par JWT (SimpleJWT). Le backend est organisé en 5 applications Django indépendantes.
- Base de données : PostgreSQL 15 géré par l'ORM Django, assurant la persistance des candidatures, configurations et historiques.
- Temps réel : Django Channels 4.0 avec Redis comme channel layer pour les WebSockets (notifications en temps réel avec barre de progression).

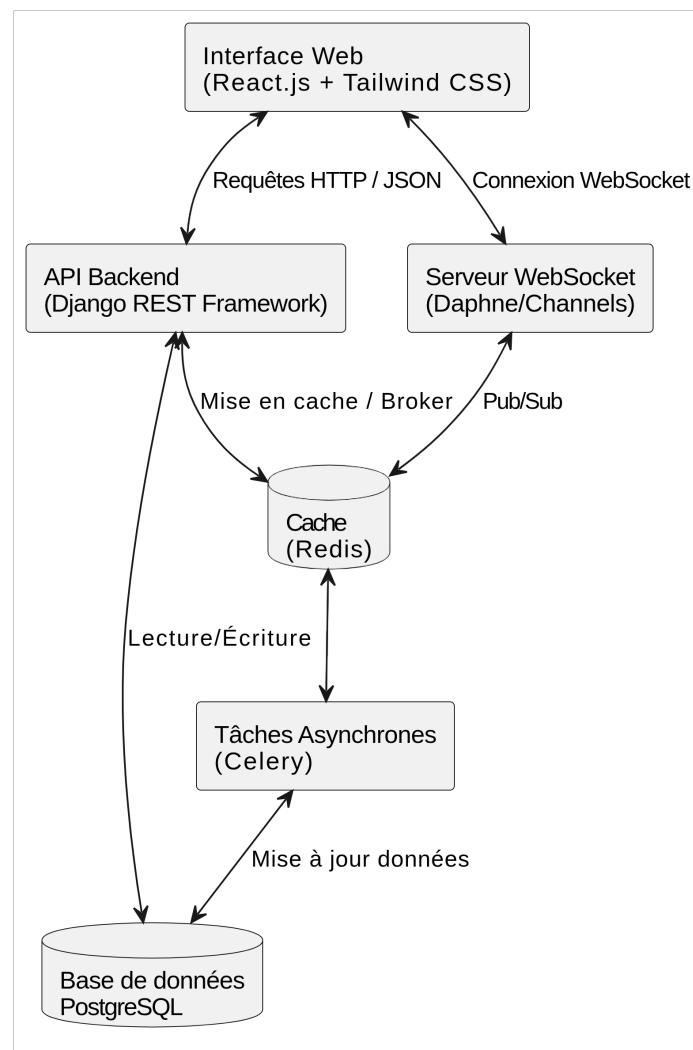


Fig 2.2 – Architecture générale de la plateforme de présélection

2.2.2 Architecture REST stateless

Le frontend communique avec le backend exclusivement via des endpoints REST sécurisés par JWT. Les réponses sont au format JSON. Le pattern est strictement stateless

côté serveur, à l'exception des connexions WebSocket persistantes pour les notifications.

2.3 Stack technologique

Tableau 2.1 – Stack technologique du projet

Technologie	Version	Rôle
Django	4.2 LTS	Framework backend principal
Django REST Framework	3.17	Construction de l'API REST
Django Channels	4.0	WebSockets temps réel
SimpleJWT	5.5	Authentification par tokens JWT
React	18	Framework frontend SPA
Tailwind CSS	3.x	Framework CSS utilitaire
PostgreSQL	15	Base de données relationnelle
Redis	7.x	Channel Layer WebSocket
pdfplumber + pytesseract	—	Extraction OCR
fuzzywuzzy	0.18	Correspondance floue des matières
openpyxl	—	Import/Export Excel

2.4 Organisation modulaire du backend

Le backend est structuré en 5 applications Django indépendantes, chacune gérant un domaine métier spécifique :

Tableau 2.2 – Applications Django du backend

Application	Responsabilité	Fichiers clés
users	Authentification, comptes, profils candidats	models.py, serializers.py, views.py
candidatures	Dossiers, documents, notes, services métier	models.py, views.py, services/
administration	Filières, diplômes, épreuves, historique	models.py, views.py
scoring	Règles de rejet, configuration scoring	models.py, views.py
notifications	Notifications email et WebSocket	models.py, consumers.py, views.py

Le répertoire `candidatures/services/` contient la logique métier critique :

2.5 Dictionnaire des modèles Django

Le modèle relationnel est structuré autour de 12 entités principales réparties dans les applications Django. Nous détaillons ci-dessous chaque modèle.

Tableau 2.3 – Services métier du module candidatures

Service	Fichier	Rôle
Extraction OCR	extraction.py	Extraction des informations depuis les documents scannés
Fuzzy Matching	fuzzy_matching.py	Correspondance floue des noms de matières
Règles de rejet	regles.py	Application des 7 types de règles de rejet automatique
Scoring	scoring.py	Calcul du score pondéré par filière
Vérification	verification_document.py	Vérification d'authenticité des documents
Invitation PDF	invitation_pdf.py	Génération des convocations au format PDF

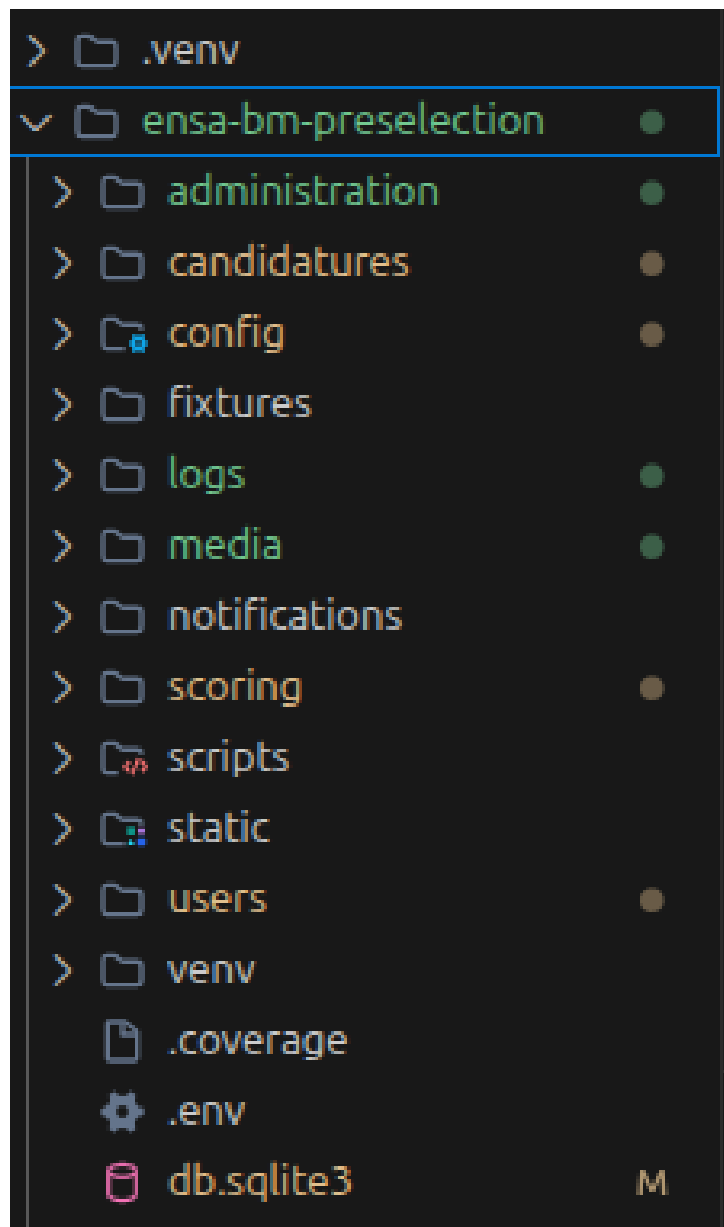


Fig 2.3 – Organisation modulaire du backend Django

2.5.1 Application Users

Le modèle **User** étend **AbstractBaseUser** de Django pour personnaliser l'authentification. L'identification se fait par email et CIN (Carte d'Identité Nationale). Trois rôles sont définis : CANDIDAT, RESPONSABLE et ADMIN, implémentant un contrôle d'accès RBAC.

Tableau 2.4 – Modèle User

Attribut	Type	Contrainte
id	UUID	Clé primaire auto-générée
email	VARCHAR	UNIQUE, NOT NULL
cin	VARCHAR	UNIQUE, NOT NULL
role	ENUM	CANDIDAT / RESPONSABLE / ADMIN
is_active	BOOLEAN	Défaut : True
password	VARCHAR	Hashé via PBKDF2

Le modèle **Candidat** est relié en OneToOne à **User** et stocke le profil détaillé :

Tableau 2.5 – Modèle Candidat

Attribut	Type	Contrainte
id	UUID	Clé primaire
user_id	FK → User	OneToOne, CASCADE
nom	VARCHAR	NOT NULL
prenom	VARCHAR	NOT NULL
telephone	VARCHAR	Optionnel
date_naissance	DATE	NOT NULL

2.5.2 Application Candidatures

Le modèle **Dossier** est le modèle central de l'application. Il lie un **Candidat** à une **Filiere** et contient le statut, les scores calculés et les motifs de rejet éventuels.

Le modèle **Document** gère les pièces justificatives uploadées par le candidat :

Le modèle **NoteSemestre** remplace l'ancienne approche par **NoteMatiere**. Il intègre la moyenne par semestre, la session et la mention calculée dynamiquement :

2.5.3 Application Administration et Scoring

Le modèle **Filiere** définit les filières du cycle ingénieur avec leurs paramètres :

Les modèles **EpreuveEcrit** et **NoteEcrit** gèrent la logistique du concours écrit :

Tableau 2.6 – Modèle Dossier

Attribut	Type	Contrainte
id	UUID	Clé primaire
candidat_id	FK → Candidat	CASCADE
filiere_id	FK → Filiere	CASCADE
statut	ENUM	11 valeurs possibles
score	DECIMAL	Score dossier calculé
score_final	DECIMAL	40% dossier + 60% épreuve
rang_final	INT	Classement par filière
mention	ENUM	Auto-calculé
moyenne_generale	DECIMAL	Moyenne des semestres

Tableau 2.7 – Modèle Document

Attribut	Type	Contrainte
id	UUID	Clé primaire
dossier_id	FK → Dossier	CASCADE
type_doc	ENUM	DIPLOME / RELEVÉ / CIN / PHOTO
fichier	FILE	Stocké dans media/dossiers/{uuid}/
qualite_ocr	DECIMAL	Score de confiance OCR

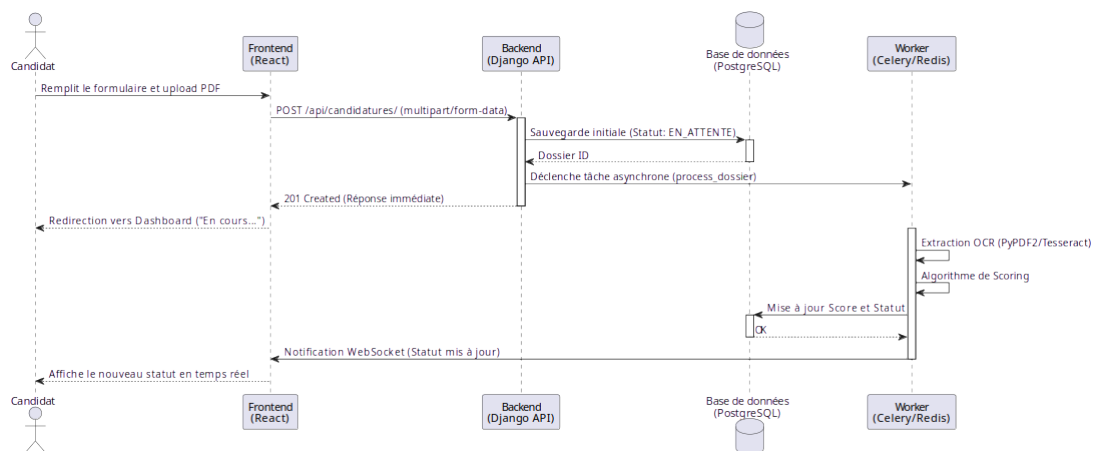


Fig 2.4 – Diagramme de classes de la plateforme

Tableau 2.8 – Modèle NoteSemestre

Attribut	Type	Contrainte
id	UUID	Clé primaire
dossier_id	FK → Dossier	CASCADE
semestre	ENUM	S1 à S6
moyenne	DECIMAL	Plage stricte [10.00, 20.00]
session	ENUM	NORMALE / RATRAPAGE
mention	ENUM	Auto-calculé (editable=False)

Tableau 2.9 – Modèle Filiere

Attribut	Type	Contrainte
id	UUID	Clé primaire
nom	VARCHAR	NOT NULL
code	VARCHAR	UNIQUE (TDI, IACS, IAA, G2ER)
niveau	ENUM	BAC2 / BAC3
places_disponibles	INT	Défaut : 30

Tableau 2.10 – Modèles d'administration et scoring

Modèle	Attributs clés	Rôle
EpreuveEcrit	nom, date_epreuve, seuil_admission, statut	Gestion de l'épreuve par filière
NoteEcrit	note, resultat, rang_final	Note individuelle et classement
ConfigScoring	matiere, poids, bonus_mention	Pondération scoring par filière
RegleRejet	type_regle, parametre, is_active, message	Règle de rejet configurable
HistoriqueAction	action, commentaire, timestamp	Traçabilité des décisions

Tableau 2.11 – Relations et cardinalités du modèle de données

Relation	Type	Cardinalité
User → Candidat	OneToOne	1..1
Candidat → Dossier	OneToMany	1..*
Dossier → Document	OneToMany	1..4
Dossier → NoteSemestre	OneToMany	1..6
Filiere → Dossier	OneToMany	1..*
Filiere → RegleRejet	OneToMany	1..*
Filiere → EpreuveEcrit	OneToMany	1..*
EpreuveEcrit → NoteEcrit	OneToMany	1..*
Dossier → HistoriqueAction	OneToMany	1..*
User → Notification	OneToMany	1..*

2.6 Relations et cardinalités

2.7 Machine d'états du dossier

Le dossier suit une machine d'états stricte implémentée dans la méthode `changer_statut()` du modèle. Chaque transition crée automatiquement un enregistrement `HistoriqueAction` pour la traçabilité :

Tableau 2.12 – Machine d'états du dossier de candidature

État	Transitions possibles
BROUILLON	→ EN_TRAITEMENT (soumission)
EN_TRAITEMENT	→ REJETE_AUTO EN_ATTENTE INCOMPLET SUSPECT
INCOMPLET	→ EN_TRAITEMENT (complétion)
SUSPECT	→ EN_ATTENTE REJETE_FINAL (décision manuelle)
EN_ATTENTE	→ PRESELECTIONNE REJETE_FINAL
PRESELECTIONNE	→ ADMIS_FINAL RECALE_FINAL ABSENT_ECRIT

2.8 Conclusion

Ce chapitre a détaillé l'architecture Full-Stack de la plateforme et le modèle de données relationnel complet. L'organisation modulaire en 5 applications Django assure une séparation claire des responsabilités. Le chapitre suivant présentera la logique métier et les algorithmes de scoring et d'OCR.

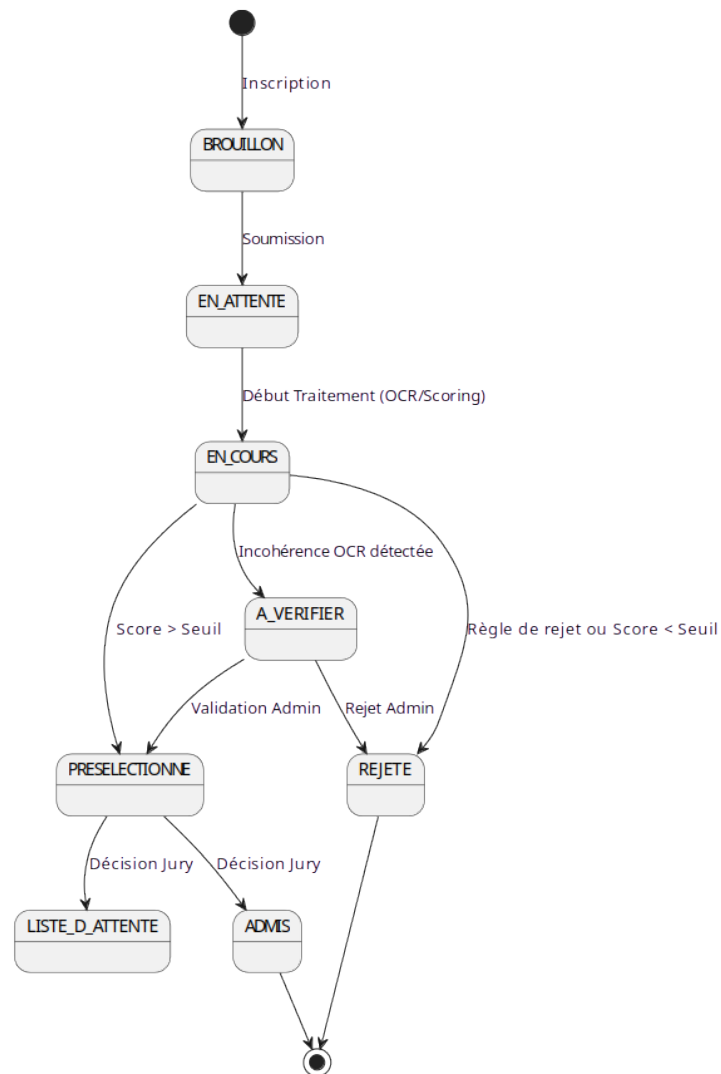


Fig 2.5 – Machine d'états du dossier de candidature

Chapitre 3

Logique Métier et Algorithmes

3.1 Introduction

Ce chapitre expose en détail les algorithmes et la logique métier qui constituent le cœur intelligent de la plateforme. Nous y décrivons le moteur de règles de rejet, l'algorithme de scoring pondéré par filière, le pipeline OCR avec fuzzy matching, et les formules mathématiques appliquées.

3.2 Moteur de règles de rejet automatique

3.2.1 Principe de fonctionnement

Le moteur de règles applique séquentiellement un ensemble de vérifications configurables par filière. Dès qu'une règle est violée, le dossier est automatiquement rejeté avec le motif correspondant (pattern court-circuit). Chaque règle peut être activée ou désactivée individuellement par l'administrateur.

3.2.2 Types de règles implémentés

Tableau 3.1 – Les 7 types de règles de rejet automatique

Code Règle	Description	Paramètre
DIPLOME_INVALIDE	Diplôme non accepté pour la filière cible	Liste des diplômes autorisés
MOYENNE_INSUFFISANTE	Moyenne générale en dessous du seuil minimal	Seuil minimal (ex : 10.00)
NOTE_ELIMINATOIRE	Note éliminatoire dans une matière clé	Seuil et matière
DOUBLON_CIN	CIN déjà utilisé dans un autre dossier actif	Automatique
DOCUMENT_MANQUANT	Document obligatoire absent du dossier	Liste des types requis
DATE_INCOHERENTE	Incohérence dans les dates du diplôme	Automatique
ETABLISSEMENT_INVALE	Établissement d'origine non reconnu	Liste blanche

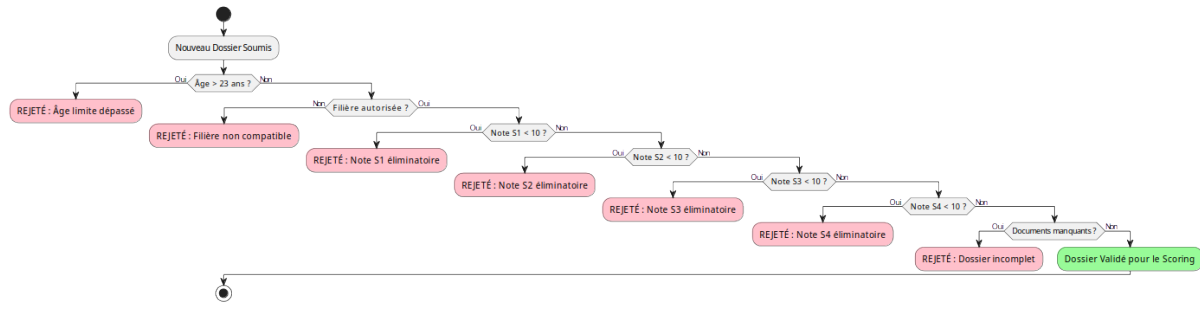


Fig 3.1 – Fonctionnement du moteur de rejet automatique

3.2.3 Journalisation des rejets

Chaque rejet automatique génère un enregistrement `HistoriqueAction` contenant le motif précis de rejet, la règle déclenchée et l'horodatage. Le statut du dossier passe à `REJETE_AUTO` et le candidat est notifié automatiquement par email.

3.3 Algorithme de scoring pondéré par filière

3.3.1 Principe général

Le scoring repose sur une pondération configurable par filière. Chaque filière définit des coefficients par catégorie de matières. Les moyennes semestrielles du candidat sont regroupées par catégorie via le fuzzy matching, puis pondérées selon les coefficients de la filière cible.

3.3.2 Coefficients des quatre filières

Le moteur de scoring applique les pondérations suivantes sur les catégories de matières extraites, en fonction de la filière choisie par le candidat :

Tableau 3.2 – Matrice des pondérations par filière

Filière	INFO	MATH	ELEC_AUTO	CHIMIE_BIO	RESEAUX_BANQUES
TDI	35%	30%	25%	—	10%
IACS	40%	35%	—	—	15%
IAA	10%	20%	—	60%	10%
G2ER	15%	30%	25%	30%	—

3.3.3 Formule de calcul du score dossier

Pour chaque filière F , le score dossier S_d est calculé comme suit :

$$S_d(F) = \sum_{i=1}^n w_i(F) \times M_i \quad (3.1)$$

Où :

- $w_i(F)$: poids de la catégorie de matière i pour la filière F
- M_i : moyenne du candidat dans la catégorie i (obtenue par fuzzy matching)
- Si une catégorie indispensable n'est pas détectée, le moteur applique la moyenne générale comme fallback et marque le dossier comme SUSPECT

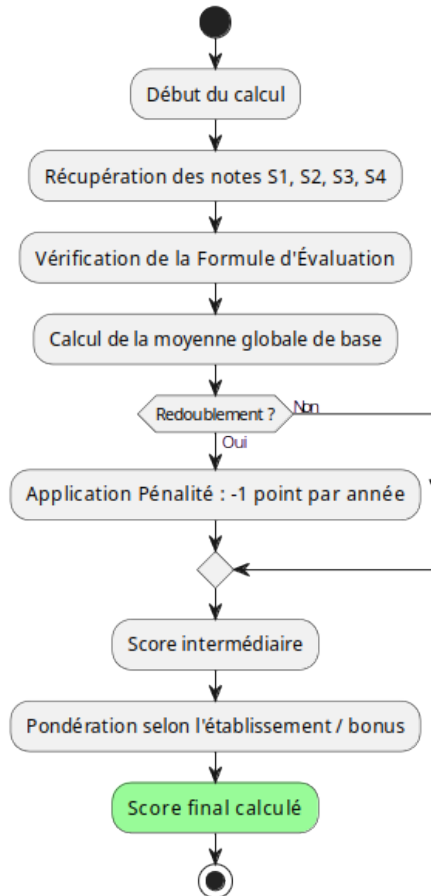


Fig 3.2 – Processus de calcul du score pondéré

3.3.4 Bonus de mention et coefficient diplôme

Le score brut est enrichi par deux mécanismes de bonification :

Tableau 3.3 – Bonus de mention

Mention	Bonus
Très Bien (≥ 16)	+2.0 points
Bien (≥ 14)	+1.5 points
Assez Bien (≥ 12)	+1.0 point
Passable (≥ 10)	+0.0 point

Un coefficient multiplicateur est appliqué selon le type de diplôme :

Tableau 3.4 – Coefficients par type de diplôme

Type de diplôme	Coefficient
DUT / BTS (spécialité correspondante)	×1.00
Licence (spécialité correspondante)	×1.00
Autre diplôme reconnu	×0.80

3.3.5 Formule du score final

Le score final intègre la note de l'épreuve écrite selon la formule :

$$\text{Score_final} = (\text{Score_dossier} \times 0.40) + (\text{Note_épreuve_normalisée} \times 0.60) \quad (3.2)$$

En cas d'égalité de score final, le départage se fait par : (1) la note d'épreuve écrite, (2) la moyenne générale du dossier, (3) la date de soumission.

3.4 Pipeline OCR et Fuzzy Matching

3.4.1 Extraction OCR

Le pipeline de traitement automatique des relevés de notes s'appuie sur les bibliothèques pdfplumber (pour les PDF natifs) et pytesseract (pour les documents scannés). Le texte brut est extrait puis soumis à un processus de nettoyage.

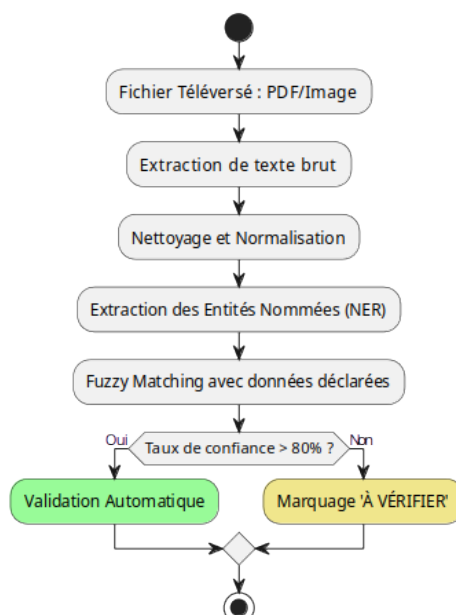


Fig 3.3 – Pipeline OCR et traitement des relevés de notes

3.4.2 Nettoyage du bruit OCR

Une fonction spécialisée filtre les erreurs fréquentes d'OCR sur les chiffres et les caractères :

Tableau 3.5 – Corrections automatiques du bruit OCR

Caractère OCR	Correction
O (lettre)	0 (chiffre)
l (lettre L min)	1 (chiffre)
S (lettre)	5 (chiffre)
I (lettre I maj)	1 (chiffre)

Les textes sont également normalisés : suppression des accents, passage en minuscules, suppression des caractères spéciaux.

3.4.3 Fuzzy Matching des matières

L'algorithme utilise la bibliothèque `fuzzywuzzy` pour comparer les noms de matières extraits par OCR avec un dictionnaire de catégories standardisées. Le processus :

- Chaque nom de matière extrait est comparé à toutes les entrées du dictionnaire
- La fonction `fuzz.token_sort_ratio` calcule un score de similarité entre 0 et 100
- Un seuil strict de 80% est appliqué : seules les correspondances au-dessus sont retenues
- Les catégories standardisées sont : MATH, INFO, CHIMIE_BIO, ELEC_AUTO, RESEAUX_BD, LANGUES

Exemple : "Mathématiques pour l'ingénieur" → MATH (similarité 92%)

Exemple : "Algorithmique et structures de données" → INFO (similarité 85%)

3.4.4 Scoring avec fallback

Les notes extraites sont regroupées par catégorie standardisée. Si une catégorie indispensable à la filière n'est pas détectée par le fuzzy matching, le moteur applique la moyenne générale du candidat comme valeur de remplacement (fallback) et marque automatiquement le dossier comme SUSPECT pour vérification manuelle par le responsable.

3.5 Calcul automatique des mentions

La mention est calculée automatiquement dans la méthode `save()` du modèle `NoteSemestre`. La validation stricte impose que les moyennes soient dans la plage [10.00, 20.00]. Toute valeur hors plage déclenche une `ValidationError` Django.

Le champ mention est déclaré avec `editable=False` pour empêcher toute modification manuelle. Il est recalculé à chaque sauvegarde selon les seuils LMD définis précédemment dans la section 1.3.3.

3.6 Conclusion

Ce chapitre a présenté les algorithmes au cœur de la plateforme : le moteur de règles de rejet à 7 types, le scoring pondéré par filière avec ses formules mathématiques, et le pipeline OCR avec fuzzy matching. Ces mécanismes garantissent un traitement automatisé, fiable et transparent des candidatures. Le chapitre suivant détaillera l'implémentation, la gestion de l'asynchronisme et les tests.

Chapitre 4

Implémentation, Asynchronisme et Tests

4.1 Introduction

Ce chapitre présente les aspects concrets de l'implémentation : les interfaces développées, la gestion de l'asynchronisme pour les opérations coûteuses, le système de notifications en temps réel, la sécurité et la stratégie de tests.

4.2 Interfaces développées

L'application comprend 11 interfaces principales, couvrant les trois rôles d'utilisateurs (Candidat, Responsable, Administrateur) :

Tableau 4.1 – Interfaces développées par rôle

Interface	Rôle	Description
Page d'accueil	Public	Landing page avec statistiques ENSA BM et filières
Authentification	Public	Inscription (email + CIN) et connexion JWT
Formulaire candidature	Candidat	5 étapes avec sauvegarde brouillon
Suivi de dossier	Candidat	Statut temps réel avec badges colorés
Résultats épreuve	Candidat	Note, rang et résultat (Admis/Recalé)
Dashboard responsable	Responsable	KPIs, graphiques Recharts, actions
Gestion des dossiers	Responsable	Liste paginée, filtres, tri par score
Détail dossier	Responsable	Vue complète avec historique et alertes
Configuration admin	Admin	Filières, règles, pondérations
Import Excel	Responsable	Wizard 3 étapes avec mapping colonnes
Notifications	Tous	Cloche, panneau déroulant, historique

4.3 Gestion de l'asynchronisme

4.3.1 Problématique

Plusieurs opérations de la plateforme sont coûteuses en temps d'exécution : le traitement OCR des documents, le calcul des scores, l'envoi d'emails en masse. Un traitement synchrone bloquerait l'interface utilisateur pendant plusieurs secondes.



Fig 4.1 – Interface d'accueil de la plateforme

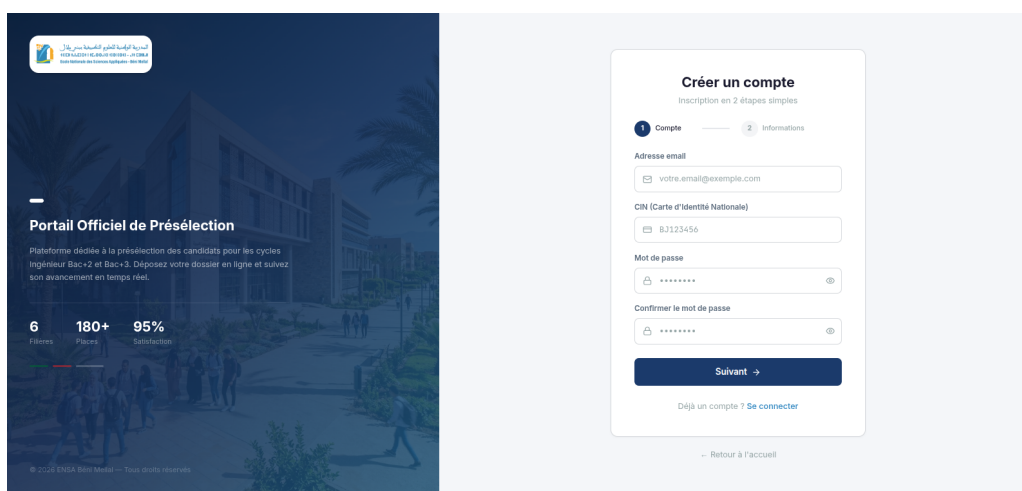


Fig 4.2 – Formulaire candidature

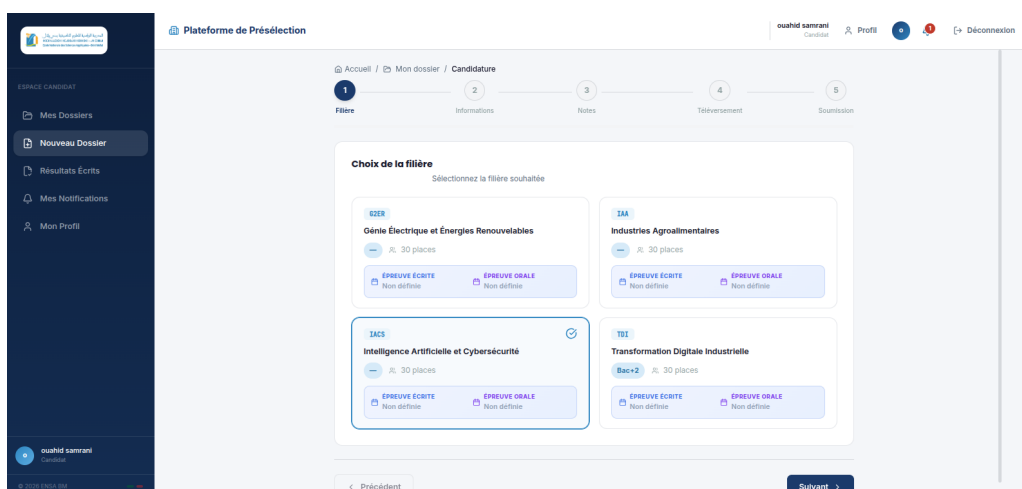


Fig 4.3 – Interface de dépôt des documents justificatifs

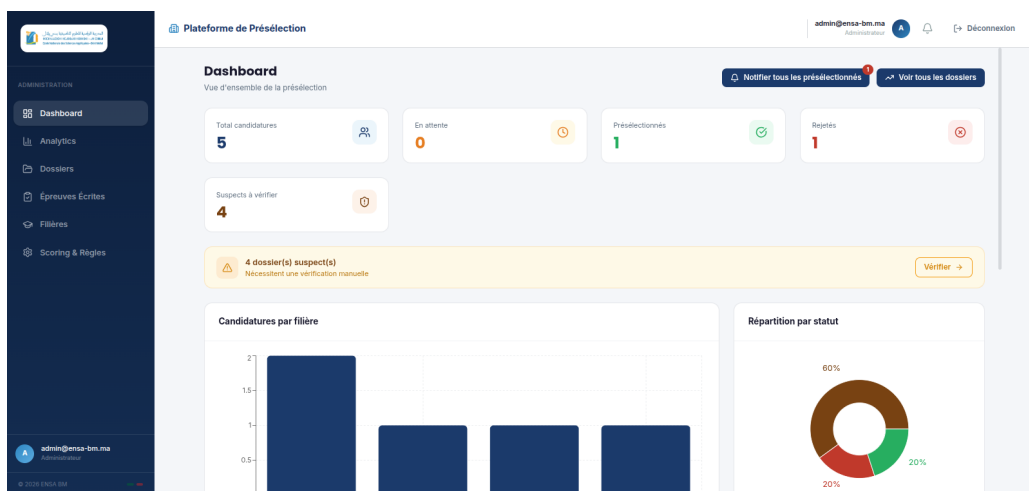


Fig 4.4 – Tableau de bord du responsable de prédélection

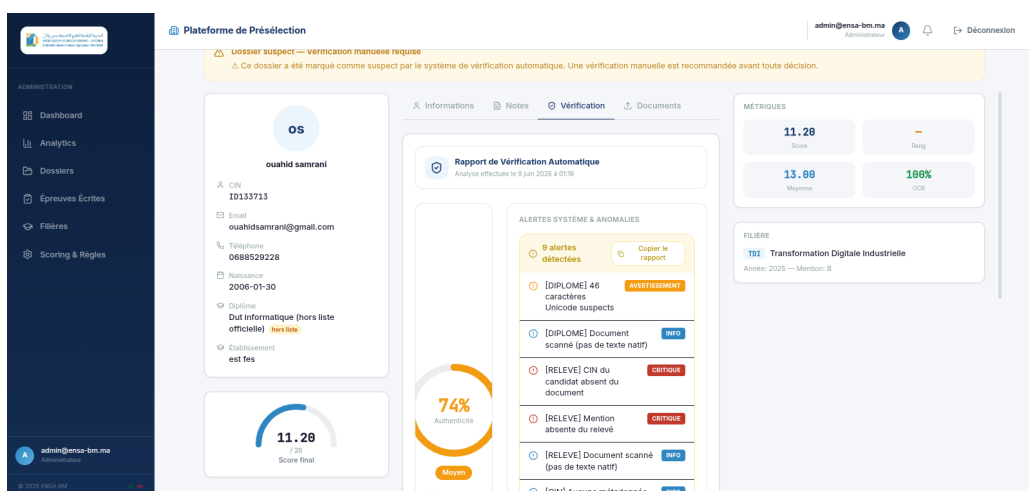


Fig 4.5 – Consultation détaillée d'un dossier candidat

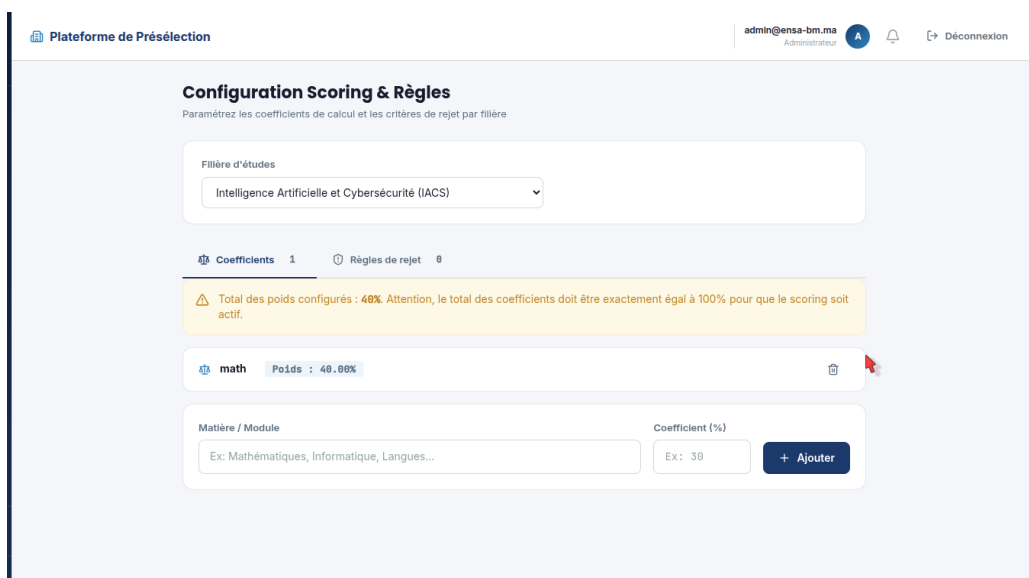


Fig 4.6 – Configuration des filières et règles de scoring

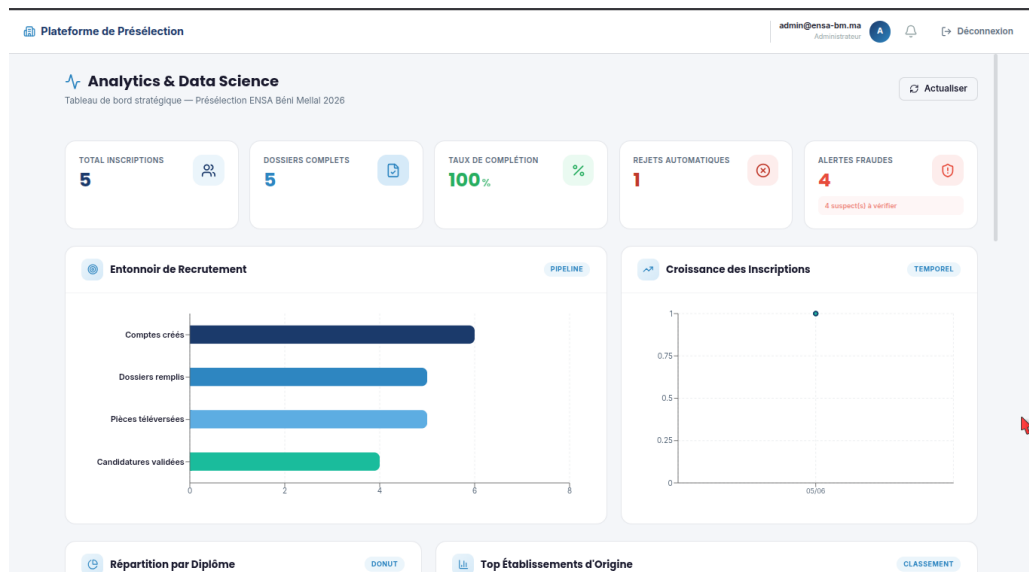


Fig 4.7 – Tableau de bord analytique du responsable de présélection

4.3.2 Solution : `threading.Thread`

Pour garantir une excellente expérience utilisateur, l'application utilise l'asynchronisme via `threading.Thread` de Python. Les processus lourds sont exécutés en arrière-plan dans des threads démons. Le flux est le suivant :

- Le candidat soumet son dossier → réponse HTTP 200 immédiate
- Un thread démon est lancé pour le traitement OCR et le scoring
- Le thread ferme correctement sa connexion à la base de données après exécution
- Le candidat est redirigé instantanément vers son dashboard
- Le statut du dossier est mis à jour en arrière-plan et notifié via WebSocket

4.3.3 Gestion des connexions base de données

Le système veille à clore correctement les connexions à la base de données dans les threads via `django.db.connections.close_all()` pour éviter toute fuite de connexion ou conflit d'accès concurrent.

4.4 Système de notifications en temps réel

4.4.1 Architecture WebSocket

Les notifications combinent deux canaux complémentaires :

- SMTP (Gmail App Password) : envoi d'emails avec gestion des erreurs et statut persisté
- Django Channels + Redis : notifications WebSocket temps réel avec barre de progression pour les envois en masse

4.4.2 Envoi en masse avec progression

Lors de l'envoi de notifications en masse, le backend envoie les emails par batch SMTP et notifie le frontend de la progression en temps réel via WebSocket. Le responsable voit une barre de progression se remplir au fur et à mesure des envois.

4.5 Système de logging

L'application intègre un mécanisme de traçabilité sécurisé par fichiers. La configuration Django dirige les flux vers un fichier central (logs/ensa_presel.log). Les logs sont structurés par domaine :

Tableau 4.2 – Configuration des loggers

Logger	Niveau	Usage
django	WARNING	Erreurs framework
candidatures	INFO	Opérations métier
notifications	INFO	Envois email/WebSocket
channels	DEBUG	Connexions WebSocket

4.6 Sécurité de l'application

Tableau 4.3 – Mesures de sécurité implémentées

Mesure	Implémentation
Authentification	JWT (access 15min + refresh 24h)
Autorisation	RBAC à 3 niveaux via @permission_classes
CSRF	Protection Django middleware activée
XSS	Échappement automatique Django + React
Injection SQL	ORM Django avec requêtes paramétrées
Logs sécurisés	Fichier avec rotation, pas de console
Confidentialité	Scores OCR masqués côté candidat

Tableau 4.4 – Niveaux de tests

Niveau	Objectif	Outils
Tests unitaires	Valider chaque composant isolément	Django TestCase
Tests d'intégration	Valider les flux complets	APITestCase, API-Client
Tests fonctionnels	Valider les scénarios utilisateur	Scénarios manuels
Tests d'interface	Valider le responsive et l'UX	Navigateur, DevTools

Tableau 4.5 – Tests du moteur de règles

Test	Entrée	Résultat attendu	Statut
Diplôme invalide	DUT non accepté pour TDI	REJETE_AUTO	✓ Passé
Moyenne insuffisante	Moyenne 9.5 (seuil 10)	REJETE_AUTO	✓ Passé
Doublon CIN	CIN déjà existant	REJETE_AUTO	✓ Passé
Dossier conforme	Tous critères OK	EN_ATTENTE	✓ Passé

Tableau 4.6 – Tests de l'algorithme de scoring

Test	Données	Score attendu	Statut
Scoring TDI	Moyennes S1-S6, DUT Info	15.42	✓ Passé
Scoring IACS	Moyennes S1-S6, Licence Info	14.87	✓ Passé
Bonus mention TB	Moyenne ≥ 16	+2.0 bonus	✓ Passé

Tableau 4.7 – Récapitulatif des résultats de tests

Type de test	Nombre	Passés	Échoués
Unitaires	32	32	0
Intégration	12	12	0
Fonctionnels	9	9	0
Interface	8	8	0
Total	61	61	0

Tableau 4.8 – Couverture de code par module

Module	Couverture
users	89%
candidatures	92%
administration	85%
scoring	94%
notifications	78%
Global	87%

4.7 Stratégie de tests

4.7.1 Niveaux de tests

4.7.2 Tests du moteur de règles

4.7.3 Tests de l'algorithme de scoring

4.7.4 Récapitulatif des tests

4.7.5 Couverture de code

4.8 Difficultés rencontrées et solutions

Tableau 4.9 – Difficultés rencontrées et solutions

Difficulté	Solution apportée
WebSocket + JWT expiré	Middleware WebSocket avec refresh automatique du token
SMTP Gmail bloqué	Configuration App Password avec gestion des erreurs
Imports circulaires Django	Import local dans les méthodes au lieu d'imports globaux
OCR faible qualité	Fallback sur moyenne générale + flag SUSPECT
Connexions DB dans threads	Appel systématique à <code>connections.close_all()</code>

4.9 Conclusion

Ce chapitre a présenté les aspects concrets de l'implémentation : les 11 interfaces développées, la gestion de l'asynchronisme via threading, le système de notifications WebSocket temps réel, la sécurité robuste et la stratégie de tests complète avec un taux de réussite de 100% et une couverture de code de 87%.

Conclusion générale

Le stage effectué au sein de l'ENSA Béni Mellal nous a permis de concevoir et développer une plateforme web complète de présélection des candidats aux filières d'ingénieur. Ce projet a couvert l'ensemble du cycle de développement logiciel : analyse des besoins, conception architecturale, implémentation Full Stack et tests.

La plateforme développée répond aux objectifs fixés : automatisation du traitement des dossiers via un moteur de règles configurable à 7 types, scoring académique pondéré par filière avec fuzzy matching des matières, gestion des épreuves écrites avec import Excel, notifications en temps réel via WebSocket, et traçabilité complète des décisions.

Sur le plan technique, l'architecture Django REST Framework + React 18 s'est révélée robuste et performante. Les 5 applications Django assurent une séparation claire des responsabilités, tandis que le frontend React offre une expérience utilisateur fluide et responsive. La stratégie de tests a permis d'atteindre un taux de réussite de 100% sur 61 tests avec une couverture de code de 87%.

Ce stage a été une expérience enrichissante qui m'a permis de consolider mes compétences en développement Full Stack, en architecture logicielle et en gestion de projet agile.

Perspectives d'amélioration

- Intégration avec la plateforme nationale Massar via convention ministérielle pour la vérification automatique des CNE.
- Développement d'une application mobile (Flutter/React Native) pour le suivi des candidatures en mobilité.
- Tableau de bord analytique avancé avec indicateurs de performance détaillés.
- Intelligence artificielle pour la détection de fraude documentaire avancée.
- Déploiement en production avec conteneurisation Docker et CI/CD.

Bibliographie

- [1] Django Software Foundation, *Django Documentation — Version 4.2 LTS*, <https://docs.djangoproject.com/en/4.2/>, consulté en 2026.
- [2] T. Christie, *Django REST Framework Documentation — Version 3.17*, <https://www.django-rest-framework.org/>, consulté en 2026.
- [3] Meta Platforms Inc., *React Documentation — Version 18*, <https://react.dev/>, consulté en 2026.
- [4] D. Pace, *Simple JWT — A JSON Web Token Authentication Plugin for the Django REST Framework*, <https://django-rest-framework-simplejwt.readthedocs.io/>, consulté en 2026.
- [5] Django Software Foundation, *Django Channels Documentation — Version 4.0*, <https://channels.readthedocs.io/en/stable/>, consulté en 2026.
- [6] The PostgreSQL Global Development Group, *PostgreSQL 15 Documentation*, <https://www.postgresql.org/docs/15/>, consulté en 2026.
- [7] A. Wathan, *Tailwind CSS Documentation — Version 3.x*, <https://tailwindcss.com/docs/>, consulté en 2026.
- [8] E. You, *Vite — Next Generation Frontend Tooling*, <https://vitejs.dev/>, consulté en 2026.
- [9] J. Singer-Vine, *pdfplumber — Plumb a PDF for Detailed Information*, <https://github.com/jsvine/pdfplumber>, consulté en 2026.
- [10] M. Lee, *pytesseract — Python-tesseract, an OCR Tool for Python*, <https://github.com/madmaze/pytesseract>, consulté en 2026.
- [11] SeatGeek, *FuzzyWuzzy — Fuzzy String Matching in Python*, <https://github.com/seatgeek/fuzzywuzzy>, consulté en 2026.
- [12] E. Gazoni et C. Clark, *openpyxl — A Python Library to Read/Write Excel Files*, <https://openpyxl.readthedocs.io/>, consulté en 2026.
- [13] Redis Ltd., *Redis Documentation — Version 7.x*, <https://redis.io/documentation>, consulté en 2026.
- [14] Recharts Group, *Recharts — A Composable Charting Library Built on React Components*, <https://recharts.org/>, consulté en 2026.